

EduPilot Agent 프로젝트 기획안

AI 기반 교육 운영 보조 멀티 에이전트 시스템 (Multi-Agent System)

"강사의 두 번째 뇌 (Second Brain)"

본 프로젝트는 강의 자료 검색, 학생 상담 관리, 과제 피드백, 평가 일정 조회 및 공지문 작성을 통합하여, 교육 현장에서 강사가 직면하는 반복적이고 소모적인 행정·운영 업무를 자동화하는 AI 기반 교육 운영 비서 시스템입니다.

1. 프로젝트 개요 및 배경

1.1 추진 배경

- 교육 현장의 피로도 증가:** 강사는 강의 외에도 학생 상담, 출결 관리, 과제 피드백, 공지 사항 작성 등 수많은 행정 업무를 동시에 수행해야 하므로 교수 효율성이 저하됨.
- 데이터의 파편화:** 강의 자료(PDF), 학생 정보 및 출결(DB), 학사 일정(Schedule) 등이 서로 다른 형태로 분산되어 있어 필요한 정보를 빠르게 취합하기 어려움.
- 해결 방안:** 각 도메인(RAG, SQL, 코드 분석, 공지 생성)에 특화된 **전문 AI 에이전트**들을 유기적으로 결합하고, 이를 중앙에서 제어하는 **Supervisor 에이전트**를 도입하여 강사의 업무를 혁신적으로 보조하고자 함.

1.2 핵심 목표

- 행정 업무 자동화:** 반복적인 조회 및 문서 작성 업무의 자동화율 80% 이상 달성.
- 멀티 에이전트 아키텍처 구현:** 단일 LLM의 한계를 극복하고, 역할 기반 분업 구조를 통한 답변 정확도 향상.
- 비동기 처리를 통한 성능 최적화:** 다중 데이터 조회 시 병렬 처리(**asyncio**)를 적용하여 데모 및 실무 환경에서의 응답 속도 최적화.

2. 시스템 아키텍처 (System Architecture)

시스템은 사용자의 자연어 입력을 분석하는 중앙 컨트롤러(Supervisor)와 분야별 임무를 수행하는 4개의 전문 에이전트 (Worker Agents)로 구성된 **Hub-and-Spoke 멀티 에이전트 구조**를 가집니다.

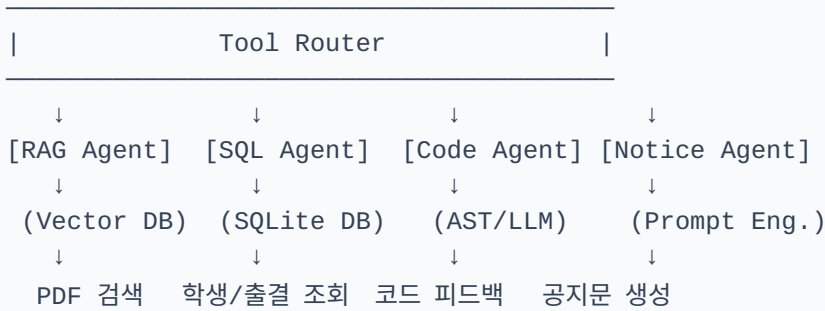
[사용자 입력 (Natural Language)]

↓

[Supervisor Agent]

↓

(Intent 분석 및 분기)



2.1 아키텍처 핵심 메커니즘

- **Supervisor Agent:** 사용자의 발화에서 의도(Intent)를 파악하고, **Tool Router** 를 통해 적절한 서브 에이전트에게 태스크를 할당 및 제어합니다.
- **비동기 콰이어링 (Async Queuing):** 여러 에이전트의 협업이 필요한 복합 질의의 경우, 비동기 병렬 처리 (**asyncio.gather**)를 통해 전체 LLM 파이프라인의 Latency를 최소화합니다.

3. 에이전트(Agent) 세부 설계

A. Lecture Agent (RAG 기반 강의 자료 검색)

- **역할:** 교재 및 교육 과정 가이드라인 문서(PDF) 내에서 정답 및 출처를 정확하게 찾아 답변합니다.
- **대상 데이터:** **React.pdf**, **JS.pdf**, **Python.pdf**, **NCS_평가기준.pdf** 등

PDF 문서 → **Chunking (Recursive)** → **Embedding (OpenAI/HuggingFace)** → **VectorDB (Chroma)** → **Retrieval** → **Generation (Answer + Source)**

- **주요 기술 Stack:** LangChain, Chroma VectorDB

B. Student Agent (Natural Language to SQL)

- **역할:** 자연어로 들어오는 학생 데이터 관리 및 출결 관련 질문을 SQL 쿼리로 변환하여 DB를 조회합니다.

- **사용 기술:** SQLite, LangChain **SQLDatabaseChain**

- **조회 시나리오 예시:**

◦ 질문: "민수 출결 상태 보여줘" → **SELECT attendance FROM students WHERE name='민수';**

◦ 질문: "취업 준비 중인 학생 누구야?" → **SELECT name FROM students WHERE career_goal='취업';**

C. Assignment Review Agent (코드 리뷰 에이전트)

- **역할:** 학생이 제출한 소스코드(텍스트 또는 파일 형태)를 분석하여 품질 향상을 위한 피드백을 제공합니다.
- **기능 범위:** 코드 설명, 구문 오류/안티 패턴 탐지, 성능 개선안 제안, 구현 난이도 평가
- **향후 확장성:** GitHub Repository API 연동을 통한 브랜치별 코드 자동 리뷰 기능 확장 가능.

D. Schedule Agent (학사 일정 관리)

- **역할:** 주차별 강의 주제, 과제 마감일, 시험 일정 등 학사 일정 DB를 전담 조회합니다.
- **조회 시나리오 예시:** "다음 주 평가 범위가 어떻게 되지?", "React 과제 마감일이 언제야?" 등의 질의 처리.

E. Notice Agent (공지사항 자동 생성)

- **역할:** 일정, 과제, 평가 등의 컨텍스트를 입력받아 양식에 맞는 완성형 공지문을 자동 템플릿화하여 생성합니다.
- **특징:** 실무에서 바로 복사·붙여넣기하여 알림톡이나 LMS에 게시할 수 있도록 높은 실무성 확보.

4. 핵심 기술 포인트 (발표 소구점)

4.1 복합 태스크 분해 (Task Decomposition)

Supervisor 에이전트는 단일 문장 속에 포함된 다중 명령을 하위 태스크로 쪼개어 다수의 에이전트에게 동시 명령을 내릴 수 있습니다.

- **복합 질의 예시:** "민수 상담 내용 보고 다음 주 평가 범위 포함해서 공지 써줘."
- **프로세스 분해:**
 1. `Student Agent` 호출 (민수 상담 내용 조회)
 2. `Schedule Agent` 호출 (다음 주 평가 범위 조회)
 3. `Notice Agent` 호출 (위 두 데이터를 종합하여 공지문 생성)

4.2 Async Task Parallelism (비동기 병렬 처리)

다중 에이전트 구조에서 발생할 수 있는 속도 저하 문제를 Python의 `asyncio` 라이브러리를 활용해 해결합니다.

```
# 복합 정보 요약 요청 시 에이전트 병렬 호출을 통한 대기시간 단축 예시
await asyncio.gather(
    get_attendance(student_name),
    get_score(student_name),
    get_consulting_notes(student_name)
)
```

💡 **발표 포인트:** 단순 순차 실행(Sequential) 대비 성능을 극대화하여 "멀티 에이전트 시스템의 실무 최적화 아키텍처"를 이해하고 구현했음을 증명합니다.

5. 데이터베이스 관계도 (Database Schema)

시스템 내 정보의 무결성을 유지하기 위해 Lightweight DB인 **SQLite**를 활용하여 아래와 같이 3개의 핵심 테이블을 설계합니다.

5.1 students (학생 정보 테이블)

```
CREATE TABLE students (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  name TEXT NOT NULL,  
  attendance INTEGER DEFAULT 0,  
  assignment_score INTEGER DEFAULT 0,  
  consulting_notes TEXT,  
  career_goal TEXT,  
  last_consult_date TEXT  
);
```

5.2 schedules (학사 일정 테이블)

```
CREATE TABLE schedules (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  week INTEGER NOT NULL,  
  topic TEXT NOT NULL,  
  assignment_due TEXT,  
  exam_date TEXT  
);
```

5.3 announcements (공지사항 이력 테이블)

```
CREATE TABLE announcements (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  title TEXT NOT NULL,  
  content TEXT NOT NULL,  
  created_at TEXT DEFAULT (datetime('now', 'localtime'))  
);
```

6. 사용자 인터페이스 (UI/UX) 설계안

프로젝트의 신속한 프로토타이핑과 높은 완성도를 위해 **Streamlit**을 활용하여 단일 페이지 대시보드를 구성합니다. (3일 개발 최적화 구조)

- **좌측 사이드바 (Admin & Ingestion Control)**: 학생 정보 관리 및 쿼리 조회, 새로운 강의 자료(PDF) 업로드 및 즉시 VectorDB 임베딩, 학사 일정 등록 및 수정
- **중앙 영역 (Core Interactivity)**: ChatGPT 스타일의 대화형 챗봇 인터페이스 (Supervisor Agent와 대화)
- **우측 대시보드 (Real-time Status)**: 최근 생성된 공지사항 히스토리 카드 피드 뷰, 최근 작성된 학생 상담 이력 요약 타임라인

7. 데모 시나리오 (Demonstration Scenario)

발표 시 시스템의 유기적인 멀티 에이전트 동작을 증명하기 위한 5단계 빌드업 시나리오입니다.

단계	사용자 질의 (자연어 입력)	작동 메커니즘 및 기대 효과
01	"지난주 React useEffect 내용 알려줘"	Lecture Agent (RAG) 가 동작하여 PDF에서 정확한 기술 개념 및 출처 페이지 답변.
02	"김민수 상담 내용 보여줘"	Student Agent (SQL) 가 자연어를 쿼리로 변환하여 DB 내 상담 기록 즉시 반환.
03	"이 학생 코드 피드백 줘"	Assignment Agent 가 코드의 안티 패턴을 분석해 오류와 개선점을 컴포넌트 단위로 출력.
04	"다음 주 평가 범위 공지 작성해줘"	Schedule + Notice Agent 가 연동하여 일정을 조회한 뒤, 공지문 템플릿 완성.
05	"김민수 출결/상담/점수 요약해서 학부모 상담용으로 정리해줘"	[하이라이트] <code>asyncio.gather</code> 를 통해 여러 테이블을 동시 조회한 후, Supervisor가 종합 리포트를 리팩토링하여 출력.

8. 구현 일정 (3-Day Sprint Plan)

일정	목표 태스크 (Milestones)	세부 개발 내용
DAY 1	인프라 및 단일 에이전트 구축	- SQLite DB 스키마 생성 및 기초 데이터 Dummy Insert - Chroma DB 연동 및 강의 PDF 텍스트 임베딩 파이프라인 구축 - 기본 RAG Agent 및 SQL Agent 단위 테스트 완료
DAY 2	오케스트레이션 및 고급 기능	- Supervisor Agent 라우팅 로직 설계 - <code>asyncio</code> 기반 비동기 병렬 처리 로직 구현 - Code Review Agent 및 Notice Agent 프롬프트 엔지니어링 및 결합
DAY 3	UI 통합 및 데모 패키징	- Streamlit 레이아웃(사이드바, 3단 분할 뷰) 구현 및 Agent 연동 - 전체 시나리오 기반 통합 엔드투엔드(E2E) 테스트 - 데모 영상 녹화 및 발표 장표(PPT) 최종 보완